

TP 4 : algorithmes de tris

ZFAI

Les questions marquées d'une ★ sont des questions théoriques. Elles peuvent être abordées plus tard, après le TP.

Exercice 1. Écrire une fonction Python `recherche(T,x)` qui est une traduction de l'algorithme de recherche dichotomique présenté dans le cours.

Correction

On a

```
def recherche(T,x) :
    a = 0
    b = len(T)-1
    while a<=b :
        c = (a+b)//2
        if T[c]>x :
            b = c-1
        elif T[c]<x :
            a = c+1
        else :
            return True, c
    return False, -1
```

Exercice 2. Écrire une fonction `tri_comptage(L)` prenant en argument une liste L dont les valeurs sont comprises entre 0 et 50 et qui renvoie la liste triée dans l'ordre croissante correspondante.

Correction

Voici un exemple du tri par comptage :

```
# Tri comptage
def tri_comptage(L) :
    M = [0 for _ in range(51)] # pour compter les occurrences de chaque élément dans L
    A = [0 for i in range(len(L))]
    for x in L :
        M[x] += 1
    indice = 0 # position courante dans la liste A pour placer un nouvel élément
    for i in range(len(M)) :
        for j in range(M[i]) :
            A[indice] = i
            indice += 1
    return A
```

Exercice 3. On donne le pseudo-code de l'algorithme de tri par sélection :

1. À quoi correspond la variable `indice` à l'issue de la deuxième boucle ?
2. Écrire en Python le pseudo-code correspondant au tri par sélection. Pour échanger le contenu de deux variables en Python, on peut écrire $a, b = b, a$.

Correction

1. À l'issue de la deuxième boucle, la variable `indice` contient la position du $(i + 1)^{\text{e}}$ minimum de la liste L .
2. Voici un exemple de code :

```

tri_selection(L)
/* L est une liste d'éléments à trier
n = longueur de L
pour i allant de 0 à n-1 inclus faire
    indice = i
    pour j allant de i + 1 à n-1 inclus faire
        si L[j] < L[indice] alors
            indice = j
        fin
    fin
    échanger L[i] et L[indice]
fin
*/

```

```

def tri_selection(L) :
    n = len(L)
    for i in range(n) :
        indice = i
        for j in range(i+1,n) :
            if L[j] < L[indice] :
                indice = j
        L[indice],L[i] = L[i],L[indice]
    return
# La liste est modifiée, mais il n'y a pas de valeur de retour.
# un certain nombre de fonctions Python se comporte ainsi, comme append.

```

- Exercice 4.**
1. (★) Montrer par récurrence qu'après k insertions, les k premiers éléments de la liste sont triés dans l'ordre croissant.
 2. Écrire une fonction Python `tri_insertion(L)` prenant en argument une liste L et qui trie la liste par un algorithme de tri par insertion.

Correction

1. Pour tout $k \in \{0, 1, \dots, n-2\}$, on pose :

$$P(k) : L[0] \leq \dots \leq L[k] \text{ après } k \text{ insertions}$$

- Initialisation : dans le cas de 0 insertion, on peut remarquer qu'une liste à un élément est triée. Donc $P(0)$ est vraie.
 - Hérité : soit $k \in \mathbb{N}$. Supposons que $P(k)$ est vraie. Montrons que $P(k+1)$ est vraie. On considère l'élément de rang $k+1$. Tant qu'il est plus petit que son prédécesseur, on l'échange avec son prédécesseur. Ainsi, lorsqu'on arrête de faire des échanges, l'élément est plus grand que son prédécesseur et plus petit que son successeur. Les $k+2$ premiers éléments sont alors bien triés. $P(k+1)$ est alors vraie.
 - Conclusion : P est donc vraie pour tout $k \in \{0, 1, \dots, n-1\}$.
2. Voici un exemple de tri par insertions :

```

def tri_insertion(L) :
    n = len(L)
    for i in range(n-1) :
        for j in range(i+1,0,-1) :
            if L[j] < L[j-1] :
                L[j],L[j-1] = L[j-1],L[j]
            else :
                break
    return

```

Exercice 5. Pour fusionner deux listes qui sont déjà triées, on peut utiliser l'algorithme récursif suivant :

1. Écrire une fonction `fusion(L,M)` prenant en arguments deux listes qui sont triées et qui renvoie la liste triée obtenue en fusionnant les deux listes. Par exemple, `fusion([1,3,5,6,9],[2,3,4,5,5])` renvoie `[1,2,3,3,4,5,5,6,9]`.
2. En déduire une fonction `tri_fusion(L)` qui renvoie la liste obtenue en triant L à l'aide d'un algorithme de tri fusion.

```

fusion(L,M)
/* L, M sont deux listes déjà triées */
si L ou M est la liste vide alors
| renvoyer L+M
fin
si L[0] < M[0] alors
| renvoyer [L[0]]+(fusion (L[1:],M))
fin
sinon
| renvoyer [M[0]]+fusion (L,M[1:])
fin

```

3. (★) Justifier la correction de l'algorithme de **fusion**, c'est-à-dire que l'algorithme renvoie bien une liste triée à la fin de celui-ci. Pour cela, on peut démontrer la propriété suivante par récurrence sur la somme des deux longueurs : si $\text{len}(L) + \text{len}(M) \leq n$ et que L et M sont deux listes triées, alors la fusion des deux est triée.

Correction

1. Voici la fonction **fusion** :

```

def fusion(L,M) :
    if L == [] or M == [] :
        return L+M
    if L[0] < M[0] :
        return [L[0]] + fusion(L[1:],M)
    return [M[0]] + fusion(L,M[1:])

```

2. Voici la fonction demandée :

```

def tri_fusion(L) :
    if len(L) <= 1 :
        return L
    n = len(L)//2
    L1 = L[:n]
    L2 = L[n:]
    M1 = tri_fusion(L1)
    M2 = tri_fusion(L2)
    return fusion(M1,M2)

```

3. Pour tout $k \in \mathbb{N}$, on note $P(k)$ la proposition "pour deux listes triées, la fusion de deux listes triées dont la somme des tailles est inférieure à k , alors la fusion est triée".
- Initialisation : soient L, M deux listes vides. Leur fusion est la liste vide qui est bien triée.
 - Hérédité : soit $k \in \mathbb{N}$. On suppose $P(k)$ vraie. Soient L, M deux listes triées dont la somme des tailles est inférieure à $k + 1$. Montrons que la fusion de L, M est triée.
 - Cas 1 : l'une des deux listes est vide. Dans ce cas, la valeur de retour qui est la liste non vide est bien triée.
 - Cas 2 : les deux sont non vide.
 - Si $L[0] < M[0]$. On a alors $L[0]$ qui est le plus petit élément des deux listes. De plus, par hypothèse de récurrence, $\text{fusion}(L[1:],M)$ est bien triée. Ainsi, $[L[0]]+\text{fusion}(L[1:],M)$ est triée.
 - si $L[0] \geq M[0]$, le raisonnement est similaire en échangeant les rôles de L, M .
 - Conclusion : la fusion de deux listes triées est bien triée.

Exercice 6. On propose une manière d'écrire un algorithme de tri rapide : il est récursif, fait intervenir une boucle et manipule la liste en place.

1. On considère la liste $L = [3, 5, 6, 1, 6, 7, 22, 1, 5]$. Appliquer l'algorithme donné avec **debut**=0 et **fin**=8 avant les appels récursifs. On décrira l'évolution de la liste L . Pour aider à la lisibilité, on pourra mettre en évidence les positions correspondant à **plus_petit** et **plus_grand**. Ainsi, on commence par

$[3, \underline{5}, 6, 1, 6, 7, 22, 1, \overline{5}]$

2. Justifier qu'à l'issue de la la boucle tant que et le dernier échange, le pivot est bien placé dans la liste.

3. Écrire en Python le pseudo-code donné. Comment trier complètement la liste ?
4. (★) Montrer qu'il existe des configurations où le tri rapide effectue un nombre de comparaisons de l'ordre de n^2 , où n est la taille de la liste.

```

tri_rapide_aux(L,debut,fin)
/* L est une liste, debut et fin sont des entiers correspondent à des indices de L. On cherche
   à trier la liste depuis debut à fin */
si debut < fin alors
    pivot = L[debut]
    plus_petit = debut + 1
    /* indice pour placer une valeur plus petite que le pivot */
    plus_grand = fin
    tant que plus_petit < plus_grand faire
        si L[plus_petit] > pivot alors
            échanger L[plus_petit] et L[plus_grand]
            plus_grand = plus_grand - 1
        fin
        sinon
            plus_petit = plus_petit + 1
        fin
    fin
    si pivot < L[plus_petit] alors
        échanger L[debut] et L[plus_petit-1]
        fin1 = plus_petit - 2
    fin
    sinon
        échanger L[debut] et L[plus_petit]
        fin1 = plus_petit - 1
    fin
    tri_rapide_aux (debut, fin1)
    tri_rapide_aux (fin1+2, fin)
fin

```

Correction

1. Liste initiale : $L = [3, 5, 6, 1, 6, 7, 22, 1, 5]$, $debut = 0$, $fin = 8$.

On commence par présenter la première phase d'itérations :

Itération	plus_petit	plus_grand	L
Initialisation	1	8	[3, 5, 6, 1, 6, 7, 22, 1, 5]
1	1	7	[3, 5, 6, 1, 6, 7, 22, 1, 5]
2	1	6	[3, 1, 6, 1, 6, 7, 22, 5, 5]
3	2	6	[3, 1, 6, 1, 6, 7, 22, 5, 5]
4	2	5	[3, 1, 22, 1, 6, 7, 6, 5, 5]
5	2	4	[3, 1, 7, 1, 6, 22, 6, 5, 5]
6	2	3	[3, 1, 6, 1, 7, 22, 6, 5, 5]
7	2	2	[3, 1, 1, 6, 7, 22, 6, 5, 5]

On cherche ensuite à placer le pivot :

On échange $L[0]$ et $L[2]$:

[1, 1, 3, 6, 7, 22, 6, 5, 5]

Le pivot 3 est à l'indice 2, les éléments à gauche sont < 3 , à droite > 3 .

Pour trier, il suffit alors d'effectuer :

$tri_rapide_aux(L, 0, 1)$ et $tri_rapide_aux(L, 3, 8)$

2. À l'issue de la boucle **tant que**, les indices **plus_petit** et **plus_grand** coïncident. Avant cet indice, les éléments sont plus petits que le pivot. Après, ils sont plus grands. Après le dernier échange, les éléments situés avant le pivot sont plus petit, et ceux après sont plus grands. Donc le pivot est bien placé.

3. Voici un exemple de code :

```
def tri_rapide_aux(L,debut,fin) :
    if debut < fin :
        pivot = L[debut]
        plus_petit = debut + 1 # position où on place un élément plus petit que le pivot
        plus_grand = fin      # position où on place un élément plus grand que le pivot
        while plus_petit < plus_grand :
            if L[plus_petit] > pivot :
                L[plus_petit],L[plus_grand] = L[plus_grand],L[plus_petit]
                plus_grand -= 1
            else :
                plus_petit += 1
        if pivot < L[plus_petit] :
            plus_petit -= 1
        L[debut],L[plus_petit] = L[plus_petit],L[debut]
        fin1 = plus_petit
        tri_rapide_aux(L,debut,fin1-1)
        tri_rapide_aux(L,fin1+1,fin)

def tri_rapide(L) :
    tri_rapide_aux(L,0,len(L)-1)
    return
```

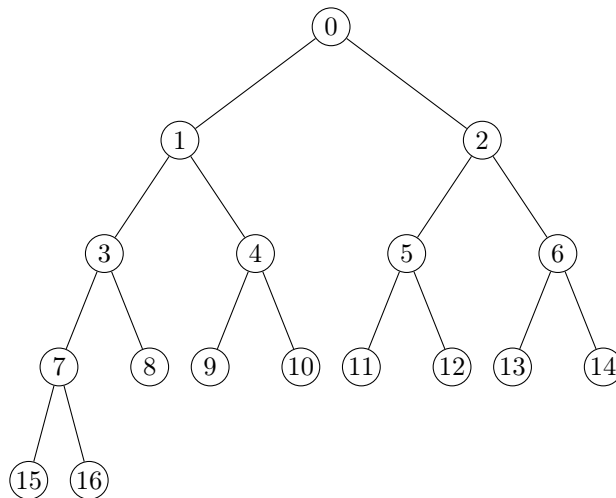
Pour les plus rapides : le tri par tas

On s'intéresse à un algorithme de tri basé sur une structure appelée tas. Il s'agit d'une liste où les éléments sont organisés d'une certaine façon.

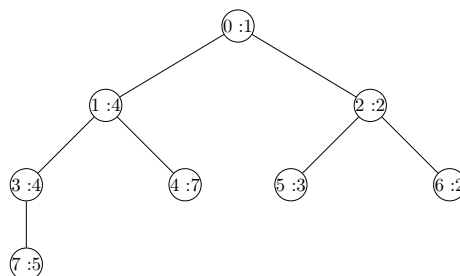
Étant donnée une liste de réels L de longueur n , on dit que L est un **tas** si la propriété suivante est vérifiée :

$$\forall i \in \{1, 2, \dots, n-1\}, L[i] \geq L[(i-1)//2]$$

Pour visualiser une structure de tas, on peut représenter la liste correspondante à l'aide d'un arbre :



les entiers écrits ici étant les indices des éléments de la liste. Représentons le tas suivant : $L = [1, 4, 2, 4, 7, 3, 2, 5]$.



dans cette représentation, dans un nœud, on a $i : L[i]$. On rappelle qu'en Python $L[i], L[j] = L[j], L[i]$ permet d'échanger les éléments d'indice i et j de L .

1. Préciser pour chacune des listes suivantes si on a un tas ou non :
 (a) $L_1 = [2, 3, 4, 2, 5, 6]$ (b) $L_2 = [4, 1, 6, 5, 3]$ (c) $L_3 = [1, 2, 2, 5, 6, 4, 3]$ (d) $L_4 = [0, 0, 1, 1, 2, 4, 6]$.
2. Écrire une fonction `est_tas(L)` prenant en argument une liste de nombres réels et qui renvoie `True` si L représente un tas et `False` sinon.
3. Donner un ordre de grandeur du nombre maximum de comparaisons de la fonction `est_tas(L)` en fonction de la taille de la liste L .
4. Écrire une fonction `diminue(L, i, a)` prenant en arguments une liste L représentant un tas, un entier i correspondant à un indice de L , un nombre réel a et qui remplace $L[i]$ par a dans le cas où $a \leq L[i]$ puis effectue des échanges pour que L garde une structure de tas. La fonction renvoie `True` si l'affectation a été effectuée et `False` sinon.
5. Écrire une fonction `augmente(L, i, a)` prenant en arguments une liste L représentant un tas, un entier i correspondant à un indice de L , un nombre réel a et qui remplace $L[i]$ par a dans le cas où $a \geq L[i]$ puis effectue des échanges pour que L garde une structure de tas. La fonction renvoie `True` si l'affectation a été effectuée et `False` sinon.
6. Écrire une fonction `ajout(L, a)` prenant en argument une liste L représentant un tas, un nombre réel a et ajoute cet élément a et qui effectue des échanges pour que L garde une structure de tas.
7. Écrire une fonction `extraire_min(L)` prenant en argument une liste non vide représentant un tas et qui renvoie le minimum de L . La fonction effectue également la modification suivante dans L : elle remplace le minimum par la dernière valeur de L , supprime le dernier élément de L et effectue des échanges pour garder la structure de tas.
8. Donner un ordre de grandeur de la complexité en nombre de comparaisons dans le pire cas de la fonction `ajout(L, a)` en fonction de la taille L .
9. Écrire une fonction `construction_tas(L)` prenant en argument une liste L et qui renvoie le tas construit obtenu en initialisant une liste T vide et qui effectue des ajouts successifs des éléments de L dans T tout en gardant une structure de tas pour T .
10. Donner un ordre de grandeur de la complexité en nombre de comparaisons dans le pire cas de la fonction `extraire_min(L)` en fonction de la taille L .
11. Pour trier une liste, on peut procéder ainsi :
 - construire le tas associé à cette liste L ,
 - extraire successivement les minimums jusqu'à ce que la liste soit vide.
- (a) Écrire une fonction `tri_par_tas(L)` qui effectue le tri décrit précédemment.
- (b) Donner un ordre de grandeur du nombre de comparaisons dans le pire cas lors du calcul de `tri_par_tas(L)` en fonction de la taille de L .

Correction

1. (a) L_1 n'est pas un tas, car $L_1[3] < L_1[1]$.
 (b) L_2 n'est pas un tas, car $L_2[1] < L_2[0]$.
 (c) L_3 est bien un tas.
 (d) L_4 est bien un tas.
2. Voici un exemple de code :

```
def est_tas(L) :
    for i in range(len(L)-1, 0, -1) :
        if L[i] < L[(i-1)//2] :
            return False
    return True
```

3. Pour une liste L de taille n , on effectue au plus $n - 1$ comparaisons.
4. Voici un exemple de code :

```
def diminue(L, i, a) :
    n = len(L)
    if i >= n or L[i] < a :
        return False
    else :
```

```

    L[i] = a
    j = (i-1)//2
    while i > 0 and L[j] > L[i] :
        L[i], L[j] = L[j], L[i]
        i = (i-1)//2
        j = (i-1)//2
    return True

```

5. Voici un exemple de code :

```

def augmente(L,i,a) :
    n = len(L)
    if i >= n or L[i] > a :
        return False
    else :
        L[i] = a
        g, d = 2*i+1, 2*i+2
        if g >= n :
            return True
        if d >= n :
            if L[g] < L[i] :
                L[g], L[i] = L[i], L[g]
            return True
        if L[g] >= a and L[d] >= a :
            return True
        if L[g] >= L[d] :
            indice = d
        else :
            indice = g
        L[i] = L[indice]

```

6. Voici un exemple de code :

```

def ajout(L,a) :
    L.append(a)
    diminue(L,len(L)-1,a)
    return

```

7. Voici un exemple de code :

```

def extraire_min(L) :
    if L == [] :
        print("liste vide")
        return
    a = L[0]
    x = L[len(L)-1]
    L.pop()
    augmente(L,0,x)
    return a

```

8. Pour une liste de taille n , la fonction ajout ajout effectue dans le pire cas k échanges, avec k vérifiant $2^k \leq n < 2^{k+1}$. Ainsi, k est égal à $\lfloor \log_2(n) \rfloor$.

9. Voici un exemple de code :

```

def construction_tas(L) :
    T = []
    for a in L :
        ajout(T,a)
    return T

```

10. Par construction, pour une liste de taille n , on obtient comme ordre de grandeur :

$$\sum_{k=1}^n \log_2(k),$$

qui est du même ordre de grandeur que $n \log_2(n)$.

11. (a) Voici un exemple de code :

```
def tri_par_tas(L) :  
    T = construction_tas(L)  
    M = []  
    for i in range(len(L)) :  
        print(M,T)  
        M.append(extraire_min(T))  
    return M
```

(b) . La complexité de la fonction `tri_par_tas` pour une liste de taille n , est obtenue ainsi : on prend en compte la construction d'un tas de taille n . Cette complexité est en $O(n \ln(n))$ d'après le calcul de la question 10. De la même façon que pour `ajout`, la complexité de `extraire_min` est en $\log_2(n)$, pour un tas de taille n . En extrayant successivement les minimums de T , on a donc une complexité de l'ordre de

$$\sum_{k=1}^n \ln(k).$$

On en déduit que la complexité est en $O(n \ln(n))$.